

JAVA spring

MOSTLY ASKED



Java

Spring Core Interview Question

01

What is Spring Framework? Why is it used?

Spring is a lightweight, comprehensive framework for Java that simplifies Java EE development. It provides support for developing Java applications through its dependency injection, aspect-oriented programming, and transaction management features. It's primarily used to make enterprise-level Java application development easier by promoting good programming practices like loose coupling through Inversion of **Control (IoC)** and **Dependency Injection (DI)**.

02

What is Dependency Injection in Spring?

Dependency Injection (DI) is a design pattern used to remove the **hard-coded dependencies** and **make the code loosely coupled**. In Spring, DI allows the Spring container to manage the dependencies of beans by injecting them either through constructor injection, **setter injection**, or **field injection**.

```
public class Car {  
    private Engine engine;  
  
    public Car(Engine engine) {  
        this.engine = engine;  
    }  
  
    public void start() {  
        engine.run();  
    }  
}
```

03

Explain the different types of Dependency Injection in Spring.

There are three types of Dependency Injection in Spring:

1. **Constructor Injection**: Dependencies are provided through the constructor.
2. **Setter Injection**: Dependencies are injected using setter methods.
3. **Field Injection (Reflection-based)**: Fields are directly injected using reflection.

```
// Setter Injection
public class Car {
    private Engine engine;

    public void setEngine(Engine engine) {
        this.engine = engine;
    }

    public void start() {
        engine.run();
    }
}
```


04

What are Spring Beans? How are they managed?

A Spring Bean is an object that is instantiated, assembled, and otherwise managed by a Spring IoC container. Beans are defined in Spring's configuration files or annotated using `@Component` or `@Bean`.

```
@Bean
public Car carBean() {
    return new Car(new Engine());
}
```

05

What is the Spring IoC container and what are the types of IoC containers?

The IoC container in Spring is responsible for instantiating, configuring, and managing the lifecycle of Spring beans. There are two types of IoC containers:

1. **BeanFactory**: It is the simplest container that provides basic support for dependency injection.
2. **ApplicationContext**: It is a more advanced container that builds on BeanFactory with additional functionalities like event propagation, declarative mechanisms to create a bean, and AOP support.

```
ApplicationContext context = new ClassPathXmlApplicationContext("beans.xml");  
MyBean bean = context.getBean("myBean", MyBean.class);
```


What is the scope of a Spring Bean?

There are five types of bean scopes available in Spring:

1. **Singleton (default)**: A single instance of the bean is created per Spring IoC container.
2. **Prototype**: A new instance is created every time the bean is requested.
3. **Request**: Scoped to an HTTP request (for web applications).
4. **Session**: Scoped to an HTTP session (for web applications).
5. **Global-session**: Scoped to a global HTTP session (for portlet applications).

07

What is the difference between @Component, @Repository, @Service, and @Controller in Spring?

These are all specialization of the @Component annotation and are used to define Spring-managed beans, but they are used for different layers in the application:

- @Component: Generic annotation for any Spring-managed component.
- @Repository: Specialization for the DAO layer. It enables exception translation.
- @Service: Indicates that the class holds business logic. Spring can apply business-specific logic here.
- @Controller: Marks a class as a Spring MVC Controller.

08

What is the difference between ApplicationContext and BeanFactory?

- **BeanFactory** is a basic container that provides dependency injection and lifecycle management of beans. It's a more lightweight option but lacks advanced features.
- **ApplicationContext** is built on top of **BeanFactory** and provides advanced features such as event propagation, declarative mechanisms to create a bean, AOP support, and internationalization.

```
ApplicationContext context = new ClassPathXmlApplicationContext("beans.xml");
```



**DON'T
FORGET TO
LIKE,
COMMENT
AND SHARE**

VISIT

WWW.TECHTRIBER.COM